

Graph-Based Agent Orchestration

Study Notes for Interview Preparation

TSAI EAG V3 — Week 9, Assignment 9
Session 8 / Session 9 Agent System

Topics covered: DAG execution, Planner role, skill isolation,
node failure, recovery replanning, orchestrator definition,
LLM context management

Prepared for interview study

Contents

1	System Overview	2
2	What is an Orchestrator?	2
2.1	Kitchen Analogy	2
3	Key Concepts	3
3.1	The Graph	3
3.2	Ready Nodes	3
3.3	Skills Catalogue	4
4	The Planner's Role	4
4.1	Common Misconception	4
4.2	What the Planner Returns	4
4.3	Three Ways the Graph Can Grow	5
5	Normal DAG Execution — Full Example	5
5.1	Initial State	5
5.2	Iteration 1 — Planner runs	5
5.3	Iteration 2 — Both Researchers Run in Parallel	6
5.4	Iteration 3 — Distiller Runs, Critic Auto-inserted	6
5.5	Iteration 4 — Critic Evaluates	7
5.6	Iteration 5 — Formatter Produces Final Answer	7
5.7	Iteration 6 — Loop Exits	7
5.8	Execution Summary	7
6	LLM Context Isolation	8
6.1	What Each Node Actually Receives	8
6.2	One Shared Element: Memory Hits	8
6.3	Why This Matters	8
7	Node Failure and Recovery	9
7.1	Recovery Philosophy	9
7.2	Failure Classification — recovery.py	9
7.3	Recovery Decision Table	9
7.4	Recovery Example — n:3 Fails	9
7.5	Final Graph After Recovery	11
8	Critic Failure Recovery	11
9	Input Resolution in Detail	12
10	Session Persistence	13
11	Interview-Ready Summary	13
12	Quick Reference: Key Files	14

System Overview

The assignment contains two major components that work together:

Component	Purpose
<code>llm_gatewayV9/</code>	A FastAPI-based multi-provider LLM routing gateway. Handles provider failover, rate limiting, streaming, tool calls, embeddings, vision, and structured output.
<code>S9SharedCode/code/</code>	A graph-based multi-skill agentic orchestrator. Decomposes user queries into a DAG of skills, executes them with dependency ordering, handles failure recovery, and maintains persistent session state.

The orchestrator calls the gateway for every LLM interaction. From the orchestrator’s perspective, the gateway is a black box: send a prompt, get text back.

What is an Orchestrator?

Definition. An **orchestrator** is the component that manages the execution of a multi-step workflow. It decides *what runs, when, and with what inputs* — without doing the actual work itself.

In this codebase the `Executor` class in `flow.py` is the orchestrator. Its job is purely mechanical:

1. Look at the graph → find which nodes are *ready*
2. Fire those nodes (hand each one to its LLM)
3. Collect results → update node statuses
4. Extend the graph if new nodes were emitted
5. Repeat until nothing is left to run

The orchestrator never reasons about the query. It does not know what “researcher” or “distiller” means semantically. It just moves work through the graph.

Kitchen Analogy

Role	Analogy
Orchestrator	Head chef calling out orders: “ <i>table 3’s starter is ready, fire the main</i> ”
Skills (LLMs)	Individual cooks — each one only knows their station
Graph	The order ticket — what depends on what
The head chef never cooks. The cooks never read other stations’ tickets.	

More formally: an orchestrator sits **above** the workers and **below** the user. It translates a high-level goal (the DAG) into a sequence of concrete, scoped calls — handling parallelism,

failure, and ordering — so that no individual worker needs to know about the bigger picture.

Key Concepts

The Graph

The agent's execution plan is a **NetworkX directed acyclic graph (DiGraph)**. Each node represents one skill invocation. Edges encode data dependencies: an edge $A \rightarrow B$ means “ B must wait for A to complete before it can run, and B will receive A 's output as input.”

Listing 1: Graph node structure (flow.py)

```

1 def add_node(self, skill: str, inputs: list[str], metadata: dict
  | None = None) -> str:
2     self._counter += 1
3     nid = f"n:{self._counter}"
4     self.g.add_node(nid, skill=skill, inputs=list(inputs),
5                     metadata=dict(metadata or {}), status="
6                             pending")
7     for inp in inputs:
8         if inp.startswith("n:") and inp in self.g.nodes:
9             self.g.add_edge(inp, nid) # edge = data dependency
10    return nid

```

Each node carries:

- **skill** — which skill type to run (researcher, distiller, etc.)
- **inputs** — list of upstream node IDs or special literals
- **status** — pending / running / complete / failed / skipped
- **metadata** — opaque dict (labels, sub-questions, failure reports)
- **result** — populated after execution (AgentResult)

Ready Nodes

A node is **ready** when all its predecessors are either **complete** or **skipped**:

Listing 2: ready_nodes() — flow.py

```

1 def ready_nodes(self) -> list[str]:
2     out = []
3     for nid, d in self.g.nodes(data=True):
4         if d["status"] != "pending":
5             continue
6         preds = list(self.g.predecessors(nid))
7         if all(self.g.nodes[p]["status"] in ("complete", "skipped")
8             for p in preds):
9             out.append(nid)
10    return out

```

Important: A failed predecessor does NOT satisfy the “complete or skipped” check. A node whose predecessor is **failed** will *never* become ready and will be left behind in the graph when the loop exits.

Skills Catalogue

Skill	Purpose	Special flags
planner	Decomposes query into DAG	Main planning entry point
researcher	Web search + fetch	<code>tools_allowed:</code> <code>[web_search, fetch_url]</code>
retriever	FAISS knowledge search	<code>tools_allowed:</code> <code>[search_knowledge]</code>
distiller	Structured extraction	<code>critic: true</code>
summariser	Condense long content	—
critic	Pass/fail evaluator	Auto-inserted by orchestrator
formatter	Final answer renderer	Terminal node
coder	Emit Python code	<code>internal_successors:</code> <code>[sandbox_executor]</code>
sandbox_executor	Run code in sandbox	Bypasses LLM entirely
browser	JS-rendered page fetching	Bypasses LLM gateway

The Planner’s Role

Key insight. The Planner emits the **entire DAG upfront** in a single LLM call — not just the next step. Every node, every branch, and every dependency edge is defined before any worker node runs.

Common Misconception

Wrong mental model	Actual behaviour
Planner → Researcher decides next → Distiller decides next → Formatter	Planner emits all nodes at once.
Each skill picks its own successor at runtime (a chain).	Workers run in dependency order. No worker decides what runs after it.
	The DAG topology is fixed by the Planner. Workers are just data processors.

What the Planner Returns

Listing 3: Typical Planner JSON output

```
{
```

```

"successors": [
  {"skill": "researcher", "inputs": ["USER_QUERY"],
   "metadata": {"label": "tokyo", "question": "Population of
Tokyo?"}},
  {"skill": "researcher", "inputs": ["USER_QUERY"],
   "metadata": {"label": "paris", "question": "Population of
Paris?"}},
  {"skill": "distiller", "inputs": ["n:tokyo", "n:paris"],
   "metadata": {"label": "distiller"}},
  {"skill": "formatter", "inputs": ["n:distiller"]}
]
}

```

All four nodes are added to the graph immediately. The Planner's job is done.

Three Ways the Graph Can Grow

These are processed in `Graph.extend_from()` after any node completes:

1. **Dynamic successors** (`result.successors`) — what the skill's LLM output explicitly requested. This is meaningful mainly for the Planner. Workers rarely emit successors.
2. **Static internal successors** (`internal_successors` in `agent_config.yaml`) — unconditional. Example: `coder` always appends `sandbox.executor`.
3. **Critic auto-insertion** — when a skill with `critic: true` (e.g. `distiller`) completes, the orchestrator automatically inserts a `critic` node between it and every outgoing child. No skill requested this — the orchestrator enforces it.

Normal DAG Execution — Full Example

Query: *"Compare the population of Tokyo and Paris"*

Initial State

```

1 graph.add_node("planner", inputs=["USER_QUERY"])
2 # Graph: n:1 planner status=pending

```

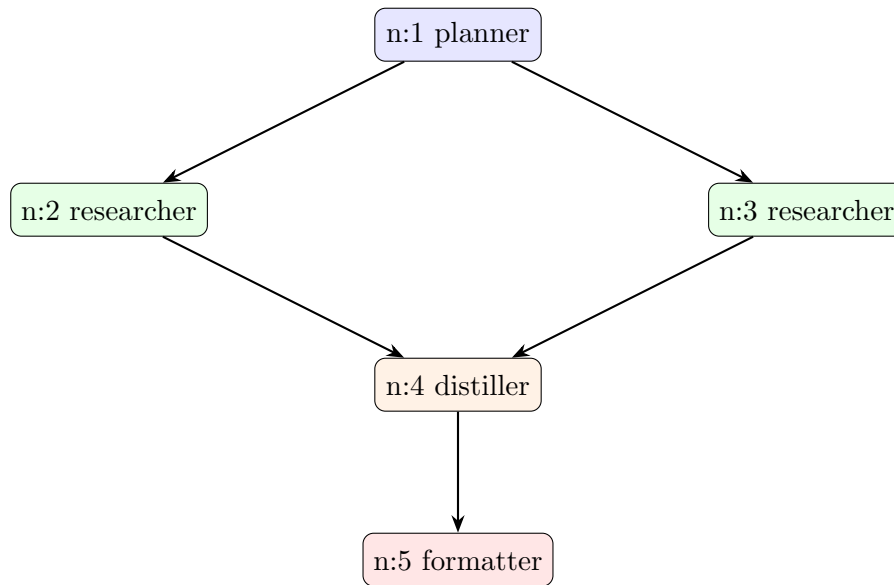
Iteration 1 — Planner runs

`ready_nodes()` returns `[n:1]` — no predecessors.

The Planner LLM receives only `USER_QUERY`. It emits four successor nodes. `extend_from()` resolves labels and builds edges:

n:1	planner	complete		
n:2	researcher	pending	inputs=[USER_QUERY]	(Tokyo)
n:3	researcher	pending	inputs=[USER_QUERY]	(Paris)
n:4	distiller	pending	inputs=[n:2, n:3]	
n:5	formatter	pending	inputs=[n:4]	

Edges: n:1->n:2, n:1->n:3, n:2->n:4, n:3->n:4, n:4->n:5



Iteration 2 — Both Researchers Run in Parallel

`ready_nodes()` returns `[n:2, n:3]` — both have only `n:1` as predecessor, which is complete.

Listing 4: Parallel execution — `flow.py:251`

```

1 outcomes = await asyncio.gather(*[
2     self._run_one("n:2", ...),    # Tokyo researcher
3     self._run_one("n:3", ...)    # Paris researcher
4 ])
    
```

They run simultaneously. Neither emits successors. Graph after:

n:2	researcher	complete	output={"findings": "Tokyo population is 13.96M..."}
n:3	researcher	complete	output={"findings": "Paris population is 2.16M..."}
n:4	distiller	pending	(still waiting)
n:5	formatter	pending	(still waiting)

Iteration 3 — Distiller Runs, Critic Auto-inserted

`ready_nodes()` returns `[n:4]` — predecessors `n:2` and `n:3` are both complete.

The distiller's rendered prompt contains the outputs of **both** researchers in its `INPUTS` block. It produces structured data.

After `extend_from()`, the critic auto-insertion fires because `distiller` has `critic: true`:

Listing 5: Critic auto-insertion — `flow.py:161–179`

```

1 # remove edge n:4 -> n:5
    
```

```

2 # insert n:6 (critic) with inputs=[USER_QUERY, n:4]
3 # add edges: n:4->n:6, n:6->n:5

```

Graph after:

```

n:4  distiller  complete  output={"tokyo_pop": "13.96M", "
      paris_pop": "2.16M"}
n:6  critic     pending   inputs=[USER_QUERY, n:4]    <- auto-
      inserted
n:5  formatter  pending   inputs=[n:4]                <- now
      gated behind n:6

```

Iteration 4 — Critic Evaluates

`ready_nodes()` returns `[n:6]`.

Critic receives `USER_QUERY` and `n:4`'s output. It emits `{"verdict": "pass"}`. `handle_critic_verdict` sees "pass" → returns `False` → normal flow continues. `n:5` becomes ready.

Iteration 5 — Formatter Produces Final Answer

`ready_nodes()` returns `[n:5]`.

Formatter receives only `n:4`'s clean structured output. It produces:

```

{"final_answer": "Tokyo (13.96M) is approximately 6x larger than
  Paris (2.16M)."}

```

The orchestrator captures this:

```

1 if graph.g.nodes[nid]["skill"] == "formatter":
2     fa = result.output.get("final_answer")
3     if isinstance(fa, str) and fa.strip():
4         formatter_answer = fa

```

Iteration 6 — Loop Exits

```

1 while True:
2     ready = graph.ready_nodes()
3     if not ready and not graph.has_running():
4         break # exits here

```

Execution Summary

Iteration	Nodes running	Key event
1	n:1 planner	Full DAG built from <code>USER_QUERY</code>
2	n:2, n:3 parallel	Both researchers fire simultaneously
3	n:4 distiller	Critic auto-inserted after completion
4	n:6 critic	Evaluates distiller; verdict = pass
5	n:5 formatter	Final answer produced
6	(none)	Loop exits

LLM Context Isolation

Each LLM call only sees the outputs of its explicitly wired input nodes — not the entire graph.

The key line is in `skills.py`:

Listing 6: Context is scoped to node inputs only — `skills.py:266`

```
1 resolved = resolve_inputs(graph_nodes[node_id]["inputs"],
2   graph_nodes, query)
3 #
#                               ~~~~~
#                               ONLY this node's input list, not all
graph_nodes
```

`graph_nodes` (the full graph) is passed in, but only `graph_nodes[node_id]["inputs"]` is resolved. Everything else is ignored.

What Each Node Actually Receives

Node	Inputs wired	Context in prompt
n:2 researcher	[USER_QUERY]	Query text only
n:3 researcher	[USER_QUERY]	Query text only
n:4 distiller	[n:2, n:3]	Outputs of n:2 and n:3 only. No USER_QUERY, no planner output
n:5 formatter	[n:4]	Output of n:4 only. Does not see raw researcher findings

One Shared Element: Memory Hits

FAISS memory hits are read **once** at session start and passed to *every* skill's prompt:

Listing 7: Memory read once, shared across all nodes — `flow.py:221`

```
1 memory_hits = memory_svc.read(query) or []
2 # ... threaded into every run_skill call
```

This is capped at 8 hits (≈ 400 chars each) — a small fixed overhead, not the whole graph.

Why This Matters

- Context stays small and predictable per node
- Parallel nodes (n:2 and n:3) have completely independent contexts — no cross-contamination
- A 10-node graph does not mean any single LLM sees $10\times$ the tokens
- Cheap small models can be used for leaf nodes; smarter models only for the Planner
- The DAG structure is the orchestrator's concern — LLMs only see their slice

Node Failure and Recovery

Recovery Philosophy

The graph is **never** serialised and passed to the recovery Planner. Instead, the orchestrator extracts two things and injects them as the recovery Planner node's inputs:

1. A short **failure report string** — what broke and why
2. The **node IDs of all already-completed nodes** — their actual output data

Failure Classification — recovery.py

Listing 8: Failure classifier — recovery.py:29–43

```

1 def classify_failure(error_text: str) -> RecoveryReason:
2     transient_markers = ("503", "502", "504", "timeout", "
        connection", ...)
3     if any(m in e for m in transient_markers):
4         return "transient"
5     if "malformed" in e or "validationerror" in e:
6         return "validation_error"
7     return "upstream_failure"

```

Recovery Decision Table

Failure reason	Action	Why
transient (503/timeout)	skip	Gateway already retried; no point replanning
validation_error	skip	Prompt bug; fix the prompt, not the run
upstream_failure + planner	skip	Re-planning a planner would cause an infinite loop
upstream_failure + other	replan	Queue a recovery Planner node

Recovery Example — n:3 Fails

Same query: “Compare the population of Tokyo and Paris” **Failure:** n:3 (Paris researcher) fails with "upstream parse error in content".

Step 1 — classify:

```

classify_failure("upstream parse error in content")
# none of the transient markers match -> "upstream_failure"

```

Step 2 — collect completed nodes:

Listing 9: Collecting prior completed nodes — flow.py:304–310

```

1 prior_complete = [
2     n for n, d in graph.g.nodes(data=True)

```

```

3     if d.get("status") == "complete"
4     and d["skill"] not in ("planner", "critic")    # skip routing
        nodes
5     and isinstance(d.get("result"), AgentResult)
6 ]
7 # prior_complete = ["n:2"]    <- only Tokyo researcher finished

```

Step 3 — add recovery Planner node:

Listing 10: Adding recovery Planner — flow.py:311–321

```

1 rec_nid = graph.add_node(
2     "planner",
3     inputs=["USER_QUERY", "n:2"],          # USER_QUERY + all
        completed outputs
4     metadata={
5         "failure_report": "node=n:3 skill=researcher reason=
            upstream_failure error=...",
6         "recovers": "n:3",
7         "prior_complete": ["n:2"],
8     }
9 )
10 # rec_nid = "n:6"

```

Step 4 — recovery Planner's rendered prompt:

Listing 11: What the recovery Planner LLM receives

```

<Planner system prompt>

USER_QUERY: Compare the population of Tokyo and Paris

FAILURE:
node=n:3 skill=researcher reason=upstream_failure error=upstream
parse error...

MEMORY HITS (N from FAISS):
...

INPUTS:
[
    {"id": "USER_QUERY", "kind": "query",    "value": "Compare the
        population..."},
    {"id": "n:2",        "kind": "upstream", "skill": "researcher",
        "output": {"findings": "Tokyo population is 13.96 million
            ..."}}]

```

The recovery Planner sees: the original query, what already worked (Tokyo research), and what failed (Paris researcher). It emits only the missing piece:

```

{
    "successors": [

```

```

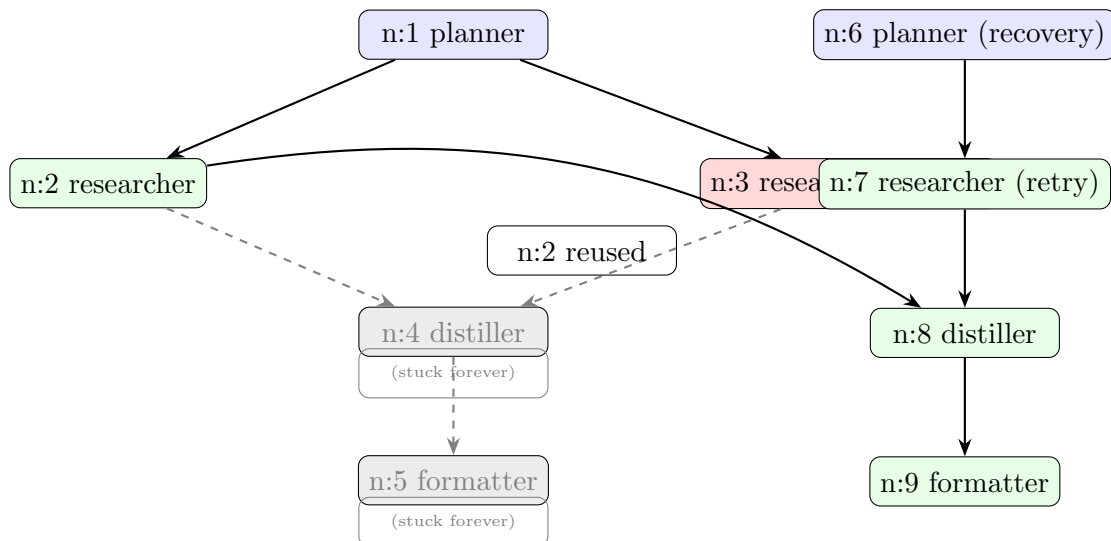
{"skill": "researcher", "inputs": ["USER_QUERY"],
 "metadata": {"label": "paris_retry", "question": "Population
  of Paris?"}},
{"skill": "distiller", "inputs": ["n:2", "n:paris_retry"]},
{"skill": "formatter", "inputs": ["n:distiller_retry"]}
]
}

```

Final Graph After Recovery

n:1	planner(original)	complete	
n:2	researcher	complete	[Tokyo data - REUSED]
n:3	researcher	failed	
n:4	distiller	pending	<- STUCK (n:3 is "failed", never "skipped")
n:5	formatter	pending	<- STUCK (waiting on n:4)
n:6	planner(recovery)	complete	
n:7	researcher	runs...	<- Paris retry
n:8	distiller	pending	
n:9	formatter	pending	

Key observation: The recovery Planner builds a **fresh parallel branch** reusing n:2's already-completed output. The stuck old nodes (n:4, n:5) are simply left behind — the loop naturally exits once the new branch finishes.



Critic Failure Recovery

The critic failure path is distinct from node failure and is handled by `handle_critic_verdict()` in `recovery.py`.

Listing 12: Critic-fail recovery — `recovery.py:97–135`

```

1 def handle_critic_verdict(nid, result, graph, recovered_branches,
  cap_hit):
2     if result.output.get("verdict", "pass") != "fail":

```

```

3         return False      # pass -> normal flow
4
5     # Mark the child node (the one the critic was gating) as
        skipped
6     graph.mark(child_nid, "skipped")
7
8     # First failure on this target: queue a recovery Planner
9     if target_nid and not recovered_branches.get(target_nid):
10         recovered_branches[target_nid] = True
11         rec_nid = graph.add_node("planner",
12                                 inputs=["USER_QUERY"],
13                                 metadata={"failure_report": fr,
14                                           "recovers": target_nid
15                                           ,
16                                           "recovery_reason": "
17                                           critic_fail"})
18     else:
19         # Already recovered once and critic failed again -> cap
20         hit, skip
21         cap_hit.append(target_nid)

```

Scenario	Behaviour
Critic says pass	Child node becomes ready; normal flow continues
Critic says fail (first time)	Child marked skipped; recovery Planner queued with inputs=[USER_QUERY]
Critic says fail (second time)	Cap hit; branch skipped entirely; final answer reflects missing data

Input Resolution in Detail

`resolve_inputs()` in `skills.py` materialises each input reference into a dict for the prompt:

Listing 13: Input resolution — `skills.py`:77–110

```

1 def resolve_inputs(node_inputs, graph_nodes, query):
2     for inp in node_inputs:
3         if inp == "USER_QUERY":
4             # literal: the original query text
5             out.append({"id": "USER_QUERY", "kind": "query", "
6                         value": query})
7
8         elif inp.startswith("n:") and inp in graph_nodes:
9             # upstream node: pull its AgentResult.output
10            upstream = graph_nodes[inp].get("result")
11            out.append({"id": inp, "kind": "upstream",
12                        "skill": upstream.agent_name, "output":
13                        upstream.output})
14
15        elif inp.startswith("art:"):

```

```

14         # artifact: bytes from content-addressable store,
           decoded as utf-8
15         blob = artifacts_svc.get_bytes(inp)
16         out.append({"id": inp, "kind": "artifact", "text":
           blob.decode(...)})
17
18     else:
19         # free-form literal: pass through unchanged
20         out.append({"id": inp, "kind": "literal", "value":
           inp})

```

Input form	What the skill receives
USER_QUERY	The original user query string
n:2	The full <code>AgentResult.output</code> dict from node n:2
art:sha256..	The raw bytes of a stored artifact, decoded to text
anything else	Passed through as a literal string

Session Persistence

Every run is associated with a session ID. The `SessionStore` in `persistence.py` writes:

- **graph.pkl** — the full NetworkX DiGraph (for resume support)
- **query.txt** — the original user query
- **nodes/{nid}.json** — per-node result snapshots

A run can be resumed with `--resume <session_id>`. The orchestrator reloads the graph, resets any running nodes back to pending (in case of a crash mid-run), and re-reads the original query.

Interview-Ready Summary

Questions you should be able to answer confidently:

1. What is an orchestrator?

A component that manages what runs, when, and with what inputs — without doing the actual work. It sits above the workers (LLMs/skills) and below the user.

2. What does the Planner do and when?

The Planner runs first, receives only `USER_QUERY`, and emits the *entire DAG upfront* — all nodes and their dependency edges — in one LLM call. It is also re-invoked on node failure (recovery) or critic failure.

3. How does the execution loop work?

While there are ready nodes (predecessors all complete/skipped), fire all ready nodes in parallel via `asyncio.gather`. Collect results, update statuses, extend the graph. Repeat until nothing is ready and nothing is running.

4. How is context kept small per LLM?

Each node's inputs list is explicit and minimal. `resolve_inputs` only materialises

what is listed — not the full graph. A 10-node graph does not mean any LLM sees 10 nodes' worth of tokens.

5. How does recovery work?

On upstream failure, the orchestrator collects prior completed node IDs and adds a new Planner node with `inputs = [USER_QUERY] + prior_complete` and a short failure report. The recovery Planner receives the failure context and the already-computed data, then emits a targeted recovery DAG. The old stuck nodes are left behind.

6. What is critic auto-insertion?

When a skill with `critic: true` (e.g. distiller) completes, the orchestrator automatically inserts a Critic node on every outgoing edge, without any skill requesting it. The Critic evaluates quality; on fail it marks the child skipped and queues a recovery Planner.

7. What happens to stuck nodes after failure?

If a node's predecessor is `failed` (not `skipped`), the node is never added to `ready_nodes` and simply stays `pending` forever. The loop exits naturally when no nodes are ready and none are running.

Quick Reference: Key Files

File	Responsibility
<code>flow.py</code>	Orchestrator (Graph + Executor). The main loop, node scheduling, critic insertion, recovery dispatch.
<code>skills.py</code>	Skill registry, input resolution, prompt rendering, LLM dispatch.
<code>recovery.py</code>	Failure classification and recovery decision logic.
<code>schemas.py</code>	<code>AgentResult</code> , <code>NodeSpec</code> , <code>NodeState</code> data models.
<code>persistence.py</code>	Session storage (graph.pkl, node snapshots).
<code>memory.py</code>	FAISS-backed memory read/write.
<code>agent_config.yaml</code>	Skill definitions, tools, critic flags, internal successors.
<code>llm_gatewayV9/main.py</code>	FastAPI gateway. Provider failover, routing, rate limiting.
<code>llm_gatewayV9/router.py</code>	Router logic: tier classification, provider selection.
<code>llm_gatewayV9/providers.py</code>	Per-provider adapter implementations.